Mid-Level Front-End Engineer (React / TypeScript)

Opportunity

We are seeking a Mid-Level Front-End Engineer to help us build, refine, and scale our core user-facing web applications. Our engineering team is currently focused on replacing legacy front-end architectures with highly responsive, modular, and optimized single-page applications. In this role, you will be instrumental in translating UI/UX designs into pixel-perfect, highly maintainable codebases that directly impact our users' day-to-day productivity and overall experience.

This is a highly hands-on front-end software engineering position tailored for a professional who thrives on modern development workflows and clean code architecture. You will spend the vast majority of your time writing production-ready TypeScript, building scalable component lifecycles, and managing client-side data synchronization. The ideal candidate is deeply opinionated about front-end performance, treats comprehensive unit testing as a vital component of delivery, and enjoys selecting and integrating modern tools to streamline our engineering ecosystem.

Requirements

- 5+ years of professional software engineering experience specializing in JavaScript and TypeScript
- 3+ years of deep, hands-on production experience building applications with vanilla React using modern build tooling like Vite
- 2+ years of experience managing server state, caching, and data fetching using TanStack Query (React Query)
- Proven experience engineering modern, accessible, and responsive user interfaces utilizing TailwindCSS and component primitives like shadcn/ui or Radix UI
- Robust experience implementing complex client-side form architectures, validation schemas, and state tracking via React Hook Form
- 2+ years of experience writing comprehensive unit and integration tests using Vitest or Jest, alongside testing utilities like React Testing Library
- Strong understanding of modern front-end build pipelines, package management, and performance optimization techniques (e.g., code-splitting, bundle analysis)

Your Responsibilities

- Develop, optimize, and maintain scalable, highly interactive user interfaces using React, TypeScript, and Vite
- Build high-fidelity, accessible components using TailwindCSS and shadcn/ui, establishing consistency across our design tokens and global component library
- Implement robust asynchronous state management and real-time client-side caching strategies via TanStack Query
- Architect and enforce highly performant form validation workflows with React Hook Form to streamline user data entry paths

- Write, maintain, and expand comprehensive unit and component-level integration test suites using Vitest or Jest to keep code coverage high and regressions low
- Partner with UI/UX designers and product owners to break down visual layouts and wireframes into technical specifications and clean feature implementations

You'll Love

- Working with a modern, blazing-fast local development environment powered by Vite and TypeScript with zero legacy build fatigue
- Leveraging a composable, unstyled-primitive approach with shadcn/ui and TailwindCSS that maximizes design flexibility without bloated UI libraries
- Relying on declarative server-state synchronization with TanStack Query, eliminating the need for boilerplate-heavy global state alternatives
- Collaborating with a technically mature team that holds code quality, strict typing, and comprehensive testing to a genuinely high standard

How You Work

- You are a pragmatic UI engineer who values technical excellence but prioritizes delivering consistent, high-fidelity value to users
- You maintain an organized, self-starting approach—you don't need continuous managerial oversight to discover bottlenecks, resolve tech debt, or ship features
- You communicate proactively and transparently, ensuring that technical limitations, dependency roadblocks, or API design conflicts are surfaced early
- You value clean, descriptive variable naming, comprehensive code documentation, and self-documenting architectures over clever, unreadable code tricks

What Success Looks Like

- Within the first 90 days, you have owned, implemented, tested, and deployed a major end-to-end front-end feature using our standard toolchain
- Client-side application performance metrics (e.g., Core Web Vitals, initial load times, bundle sizes) show measurable optimization under your engineering stewardship
- Complex client-side data layers and asynchronous workflows are visibly more predictable, debuggable, and performant due to structured TanStack Query architectures